先记四个量，公式就能串起来

距离 D、步数 N、NI 格数 G、时间 T：先分清单位，再做换算。

“步数、时间、格数”还缺距离。你想要的 dx、dy 和扫描范围都是距离。再给定每步位移、NI 点率、速度，就能把这些量互相换算。以下只讨论这份无加速段、匀速教学代码。

量	具体是什么	默认例子
距离 D，单位 μm	滑台的名义命令位移，不是表格行数	一个 X 间距：5 μm
步数 N，单位 STEP	需要多少个完整 STEP 周期	5 μm ÷ 0.5 μm/步 = 10 步
NI 格数 G，单位格	一段 STEP 串占的行数；同一行五路同时输出	10 步 × 100 格/步 = 1000 格
时间 T，单位 s	上述格数占用的输出时间	1000 格 ÷ 200000 格/s = 0.005 s

只记这四个等式

关系	公式	先记住单位
距离与步数	D = N × d；反过来 N = D / d	d = stepUm，μm/步
速度与周期	P = F × d / v	P：NI 格/步；v 必须用 μm/s
步数与格数	G = N × P	整段 STEP 串的格数
格数与时间	T = G / F；也可核对 T = D / v	F = niRate，NI 格/s

符号只为手算：d = p.stepUm；F = p.niRate；v = 1000 × p.speedMmS；P = s.stepTicks。这四式先算运动段，等待格数另加。轴与方向由列号和 DIR 决定，触发时刻由行号决定。

一条计算链：5 μm 的 X 间距怎样变成 5 ms

```
d = 0.5;  F = 200000;  v = 1000;  D = 5;
P = F*d/v;   % 100 NI ticks per STEP
N = D/d;     % 10 STEP cycles
G = N*P;     % 1000 NI ticks
T = G/F;     % 0.005 s = 5 ms
```

如何记住：步距把“距离”换成“步”；周期把“步”换成“格”；NI 点率把“格”换成“秒”。

这三种计数不同：一个 STEP 周期可占很多 NI 格；一个 X 分箱可包含很多 STEP 周期。

这里的格是 NI 输出格。ATS 采到的信号点另用 atsRate 计算，不能把两者混用。默认一格 NI 为 5 μs，而一个 ATS 采样点约为 5.56 ns。

我需要自己选哪些参数

先定间距、范围、速度；硬件标定与时钟不能靠 dx、dy 推出来。

1 分清你想改变的东西

参数类别	代码里的量	怎么选或怎么求
扫描网格	dxUm、dyUm	你希望相邻 X/Y 采样位置隔多远
扫描范围或数量	nx、ny	选范围后按下方公式换算；也可直接选数量
运动速度	speedMmS	决定相同距离走多久；本版允许正值且不超过 4 mm/s
引入与等待	startStep、settleS	本例先保留 10 步、0.02 s；不由 dx/dy 决定
已核对的基础设置	stepUm、niRate	本例沿用 0.5 μm/步、200000 格/s；不是用来随意改分辨率的旋钮

dx、dy 是采样间距，不是系统实际分辨率。它们规定“隔多远记录一个位置”。网格更密，不自动证明能分清更细的结构；实际分辨率仍取决于成像系统与测量。

2 先说清“范围”是哪一种，再求 nx、ny

你要固定的范围	本程序里的换算
X：所有完整分箱的总宽度 Lx	nx = Lx / dxUm；这份程序还会额外走 startStep 步
X：第一个到最后一个分箱起点的跨度 Sx	nx = Sx / dxUm + 1；它比完整分箱总宽度少一个 dx
Y：第一行到最后一行的跨度 Ly	ny = Ly / dyUm + 1；第一行就在 Y = 0

本程序没有 Lx、Ly 参数；先在纸上算出 nx、ny，再填进 p。**只改 dx 不改 nx，范围也会变；要保持范围，必须重算 nx。**选择一种 X 范围定义，不要混用。默认 nx=20、dx=5：完整分箱宽100 μm，首末起点跨度95 μm。

3 例子：范围不变，只把 X 网格加密一倍

目标：完整 X 宽度100 μm，Y 首末跨度2 μm，速度1 mm/s。把 dx 从5改成2.5 μm，dy仍为1 μm：

```
p = cscan_settings();
p.dxUm = 2.5;  p.dyUm = 1;  p.speedMmS = 1;
p.nx = 100 / p.dxUm;       % 40 X bins
p.ny = 2 / p.dyUm + 1;     % 3 Y rows
s = make_scan_sequence(p, "NPT");
```

结果：每个 X 间距由10步变5步；时间由5 ms变2.5 ms；分箱由20变40。完整 X 距离和速度不变，所以 TS 一行仍为100 ms，X 去程仍为210步。

先检查能否整除：dx/d、dy/d、nx、ny 都应满足代码的整数要求，P 也应接近整数且至少2。例：dx=0.25 μm而d=0.5时需要半步，本代码拒绝；速度3 mm/s时P≈33.333，本代码也拒绝。不要偷偷四舍五入。

本版另要求等待至少1 ms、X/Y相对命令行程各不超过5 mm；程序没有读取滑台实际限位。

move_table 就是把 STEP 串放进空表

它只做一段运动表；不用知道光声信号，也不会自己启动电机。

```
function table = move_table(nSteps, stepColumn, direction, p, period)
s.forward = move_table(210, 1, true, p, 100);
```

输入	一句话解释	X 去程例子
nSteps = N	重复多少次周期，决定命令步数	210
period = P	每次周期占几格，决定周期长短	100
stepColumn / direction	哪一列放 STEP、配对的 DIR 取什么值	1 / true，即 X 与本程序正向
p 中的 settleS、niRate	算前后空出几格：L = round(settleS × F)	L = 4000

四个动作：造周期 → 重复 → 建空表 → 放进去

```
pattern = [true(floor(P/2),1); false(ceil(P/2),1)];
train = repmat(pattern, N, 1);
R = 2*L + N*P + 1;
table = false(R, 5);
table(:,2) = true; table(:,4) = true;
table(:,stepColumn+1) = direction;
table(L+(1:N*P), stepColumn) = train;
```

这里用 N、P、L 简写源代码，含义不变。五列固定为 X STEP、X DIR、Y STEP、Y DIR、**采集触发**。冒号：选整列；DIR列 = STEP列 + 1。第5列先保持低。

把数组缩小，你就能看见“放进去”是什么意思

仅作数组示意：取 L=2、P=4、N=3，选 X 正向。一个周期是 [高 高 低 低]，重复3次共12格。空表共 R=2×2+3×4+1=17**行**。这是缩小示意，不是本机默认参数。

行号	1 X STEP	2 X DIR	3 Y STEP	4 Y DIR	5 触发
1-2：前等待	0	1	0	1	0
3-4 / 5-6	1 / 0	1	0	1	0
7-8 / 9-10	1 / 0	1	0	1	0
11-12 / 13-14	1 / 0	1	0	1	0
15-17：后等待与末格	0	1	0	1	0

真正写入的是：table(3:14,1)=train。其余 STEP 格不动，仍是0。每格输出时长1/F；高低不是实际电压值。

换回默认值：N=210、P=100、L=4000，R=29001；STEP串写进第1列的4001-25000行。高50格、低50格；最后额外1格不是多走一步。改方向不改周期，改次数不改每步速度。

完整扫描：四张表与触发行号

第21行起负责组装；每个参数如何传进去，都能用上一页公式算。

表	N：命令步数	STEP列 / DIR	默认行数 R
forward：X去程	startStep + nx×dx/d = 210	1 / true	29001
back：X返回	与去程相同 = 210	1 / false	29001
yNext：Y换行	dy/d = 2	3 / true	8201
yHome：Y返回	(ny-1)×dy/d = 4	3 / false	8401

四张表都用 R=2L+NP+1。返回是反向输出相同步数，不是检测原点；方向电平沿用当前程序约定。**这里只生成四个数组，实际执行顺序由 run_cscan 决定。**

触发只需要记两个索引公式

要找什么	公式	默认计算
第一个触发的行号 r0	L + startStep × P + 1	4000 + 10×100 + 1 = 5001
NPT第 j+1 个触发行号	r0 + j × pixelTicks，j=0…nx-1	5001、6001、…、24001
索引变时间	t = (r-1) / F	第5001行从25 ms开始

pixelTicks = (dx/d)×P，默认1000格。**为什么索引加1？**前面4000+1000格已经占用，下一格编号就是5001；第1格从时间零开始，所以换成时间时减1。

```
s.triggerTicks = s.firstTick + (0:p.nx-1)*s.pixelTicks;
if mode == "TS", s.triggerTicks = s.firstTick; end
s.forward(s.triggerTicks, 5) = true;
```

NPT：指定20个行号，每隔1000格写一次高。TS：只保留第5001行。第五列原来全低，因此写高一格就得到宽度1/F=5 μs的计划触发。

三个时间不要混：每步、每分箱、整张表

时间	公式	默认答案
一个 STEP 周期	P / F	0.5 ms
一个 X 分箱	pixelTicks / F = dx / v	5 ms
TS一行保留时长	nx × dx / v	100 ms（从首触发算）
整张 X 去程表输出时长	R / F	145.005 ms（包含前后等待）

20个NPT触发的首末只有19个间隔，共95 ms；20个完整分箱合计100 ms。上面均不包含软件处理和存盘时间；运动表时长还不能当作完整C-scan总耗时。

坐标只是同一组参数的另一个表达

X分箱起点 = startStep×d + j×dx = 5、10、…、100 μm。最后一格延伸到105 μm。
Y行坐标 = j×dy，j=0…ny-1 = 0、1、2 μm。这些是命令坐标，尚不是位置实测值。

preview 做什么，以及你怎样自己核对

NI 表决定何时输出；ATS 数据提供电压；preview 只做简单幅度预览。

1 两种“行列”，含义不同

数组	一行 / 一列是什么	默认大小
s.forward：NI输出表	一行是NI时刻；一列是一路控制信号	29001 × 5
block：NPT信号数据	一行是记录内采样序号；一列是一条记录	1024 × 20
block：TS信号数据	一行是分箱内采样序号；一列是一个时间分箱	900000 × 20

2 只记 preview 的三步

```
% NPT: one short record per column
block = reshape(raw, p.recordSamples, p.nx);
% TS: split the continuous row into nx time bins
block = reshape(raw, [], p.nx);
span = double(max(block,[],1)) - double(min(block,[],1));
values = span * (2*p.rangeV/65535);
```

两句 reshape 按模式二选一，不是先后都执行。**reshape 按列填数据**；max/min 的最后参数1表示在每列内部找极值。每列最后压成一个数，一次输出1×nx；ny行拼成ny×nx预览图。

公式：预览值(V) = (最大ADC码 - 最小ADC码) × 2×rangeV / 65535。
默认rangeV=4，量程为±4 V，跨度8 V。例：最大33000、最小32000，差1000码，预览值 = 1000×8/65535 ≈ 0.1221 V。

3 ATS点数要用另一个时钟算

NPT：1024点 / 180000000点/s ≈ 5.69 μs/记录。
TS：每分箱点数 = atsRate × pixelSeconds = 180000000 × 0.005 = 900000点。

preview 不是平均、包络或深度重建。TS一个分箱不等于一次激光；最大减最小也会受噪声尖峰和窗口长度影响。NPT与TS默认窗口不同，预览值不能直接定量比较。

4 自己算一遍：保持范围，改 dx 与速度

沿用d=0.5 μm/步、F=200000格/s、等待20 ms、startStep=10。设完整X宽100 μm、Y跨度2 μm，dx=2.5 μm、dy=1 μm、速度2 mm/s。

nx=_____；ny=_____；P=_____格/步；X间距=_____步
pixelTicks=_____；分箱时间=_____ms；X去程=_____步
去程表R=_____行；首触发r0=_____；首触发时间=_____ms
TS每分箱=_____个ATS点；preview每行输出_____个数

核对：40；3；50；5；250；1.25；210；18501；4501；22.5；225000；40。
改 p 后重新运行 s=make_scan_sequence(p,"NPT") 才得到新表；旧 s 不会自动变。

来源：Research-PAM / 07_MATLAB / simple_cscan_bilingual 内的 cscan_settings.m、make_scan_sequence.m、cscan_preview.m。上述是本版教学设置下的公式与数组计算，不是新硬件测量。